

Runbook Reproduksi & Operasional SmartCityApps

Dari Proses Analisis Data hingga Inferensi Server dan Aplikasi Mobile

*Pipeline Klasifikasi Multimodal CRM Jakarta
(DINOv3-large + multilingual-E5-large + CatBoost+PGS)*

Repositori	smartCityReport/
Server inferensi	FastAPI <code>serve_model.py (/health, /predict)</code>
Klien	Flutter Android <code>smart_city_reporter_app/</code>
Backend data	Supabase (PostgreSQL, Storage, Auth)
Publikasi	ngrok tunnel

Dokumen ini bersifat operasional (*runbook*); jalankan tahap secara berurutan.

Contents

1	Ikhtisar Alur	2
2	Prasyarat & Lingkungan	2
2.1	Menyiapkan virtualenv (penyajian, CPU)	2
2.2	Menyiapkan dependensi pelatihan (GPU / RunPod)	2
3	Tahap A — Analisis & Persiapan Data	3
4	Tahap B — Ekstraksi <i>Embedding</i> dengan <i>Encoder</i> Beku	3
5	Tahap C — Pelatihan Kepala CatBoost + PGS	3
6	Tahap D — Ekspor ONNX & Validasi Paritas	4
7	Tahap E — Menjalankan Server Inferensi FastAPI	4
7.1	Uji <i>health</i>	4
7.2	Uji <i>predict</i> (multipart)	4
8	Tahap F — Kontainerisasi Docker (opsional, GPU)	5
9	Tahap G — Publikasi Server via ngrok	5
10	Tahap H — Menjalankan Aplikasi Flutter	5
10.1	Alternatif tanpa ngrok	5
11	Tahap I — Verifikasi End-to-End	5
12	Troubleshooting	6
13	Lampiran — Peta Berkas Penting	6

1 Ikhtisar Alur

Runbook ini menjalankan keseluruhan sistem secara berurutan, mulai dari analisis data mentah sampai aplikasi mobile mengonsumsi hasil prediksi. Sembilan tahap utama dirangkum pada Tabel 1.

Tahap	Nama	Keluaran
A	Analisis & Persiapan Data	<code>train/val/test.csv</code> , statistik kelas
B	Ekstraksi <i>Embedding</i>	<i>cache</i> fitur citra & teks (1024-d)
C	Pelatihan Fusi CatBoost+PGS	<code>.cbm</code> model + metrik validasi
D	Ekspor ONNX & Paritas	artefak <code>artifacts/export/</code>
E	Server Inferensi FastAPI	layanan :8000 (<code>/predict</code>)
F	Kontainerisasi Docker	image <code>crm-multimodal:latest</code>
G	Publikasi ngrok	URL HTTPS publik
H	Aplikasi Flutter	APK / sesi <code>flutter run</code>
I	Verifikasi End-to-End	laporan tersimpan di Supabase

Table 1: Sembilan tahap runbook dari analisis hingga konsumsi aplikasi.

2 Prasyarat & Lingkungan

- **Python** 3.11 (server & notebook).
- **Flutter** SDK (channel stabil) + Android SDK / perangkat Android.
- **ngrok** (akun gratis cukup) untuk publikasi server.
- **Docker** (opsional, untuk jalur kontainer/GPU).
- **GPU** (opsional): pelatihan & ekstraksi *embedding* dijalankan di RunPod; inferensi dapat berjalan di CPU (mis. macOS / Apple Silicon).

Catatan platform. Pelatihan dan ekspor (Tahap A–D) dilakukan pada pod RunPod ber-GPU. Penyajian (Tahap E–I) dapat berjalan penuh di CPU lokal menggunakan subset dependensi `requirements-serve.txt`.

2.1 Menyiapkan virtualenv (penyajian, CPU)

```
cd smartCityReport
python3.11 -m venv .venv
source .venv/bin/activate
pip install -U pip
pip install -r requirements-serve.txt      # fastapi, uvicorn,
    ↪ onnxruntime (CPU),                  # transformers, catboost,
                                         ↪ pillow, numpy
```

2.2 Menyiapkan dependensi pelatihan (GPU / RunPod)

```
# Pada pod RunPod (CUDA):  
pip install -r requirements.txt # torch (CUDA), timm, dsb.  
pip install -r notebooks/requirements_runpod.txt
```

3 Tahap A — Analisis & Persiapan Data

Dataset CRM Jakarta (citra + narasi + label SKPD) dianalisis, dibersihkan, dan dibagi secara stratifikasi. Jalankan notebook dari dalam folder `notebooks/` agar `sys.path` relatif (`../src/`) ter-resolve.

```
cd smartCityReport/notebooks  
jupyter notebook # atau: jupyter lab
```

1. `01_dataset_exploration.ipynb` — distribusi 9 kelas, contoh citra, analisis ukuran gambar, distribusi 487 SKPD mentah. *Output:* `class_distribution.png`, `skpd_distribution.png`.
2. `02_preprocessing.ipynb` — normalisasi unicode, *whitespace collapsing*, pembersihan label, pemetaan 487 SKPD → 9 instansi target.
3. `03_split.ipynb` — *stratified split* berdasarkan `label_id`. *Output:* `train.csv` (43.241), `val.csv` (9.266), `test.csv` (9.266).

Validasi tahap. Pastikan proporsi label tiap kelas konsisten lintas partisi `train/val/test`, dan *class weights* (inverse frequency) tersimpan untuk dipakai CatBoost.

4 Tahap B — Ekstraksi *Embedding* dengan *Encoder* Beku

Setiap citra dan narasi dilewatkan melalui *encoder* beku untuk menghasilkan vektor 1024-dimensi; tidak ada gradien yang mengalir ke *encoder*.

1. `04_extract_image_embeddings.ipynb` — DINOv3-large (*resize* → *center crop* → normalisasi ImageNet → NCHW). *Output:* `cache image_emb.npy`.
2. `05_extract_text_embeddings.ipynb` — multilingual-E5-large (*tokenize* → *mean pooling* → L2-normalize). *Output:* `cache text_emb.npy`.

Tujuan caching. Karena *encoder* beku, *embedding* dihitung sekali lalu dipakai ulang oleh seluruh eksperimen pelatihan CatBoost, sehingga eksperimen menjadi cepat dan reproducible.

5 Tahap C — Pelatihan Kepala CatBoost + PGS

Embedding citra dan teks dikonkatenasi (*early fusion*) menjadi vektor 2048-d, lalu CatBoost dilatih untuk meminimalkan *multiclass logloss*.

1. `06_fusion_catboost_pgs.ipynb` — pelatihan CatBoost + kalibrasi *Posterior Gaussian Sampling*; pemilihan model terbaik berbasis *macro F1* pada set validasi. *Output:* `artifacts/models/best_catboost`
2. `07_error_analysis.ipynb` / `07_standalone_fusion_analysis.ipynb` — *confusion matrix*, analisis kesalahan, *threshold trade-off*.
3. `08_add_dinov3_ablation.ipynb`, `09_advanced_classifiers.ipynb` — ablasi modalitas dan pasangan *encoder* ($4 \times 4 \times 2$).

Hasil rujukan. Konfigurasi terbaik mencapai *balanced accuracy* 0,8074 dan *macro F1* 0,7747 pada 9.266 sampel uji.

6 Tahap D — Ekspor ONNX & Validasi Paritas

1. `08_export_onnx.ipynb` — ekspor *image encoder*, *text encoder*, dan *classifier head* ke ONNX.

Artefak final yang harus tersedia sebelum penyajian:

```
artifacts/export/
  image_encoder_dinov3_large.onnx           # 1.15 GB
  image_encoder_dinov3_large/preprocessor_config.json
  text_encoder_mE5_large/model.onnx (+ model.onnx_data ~6.4 GB)
  tokenizer/
  classifier_dinov3_large_mE5_large.onnx     # CatBoost-as-ONNX
```

Validasi paritas. Bandingkan probabilitas keluaran PyTorch vs ONNX pada sampel uji; selisih absolut maksimum harus di bawah ambang 10^{-5} agar prediksi top-1 identik.

7 Tahap E — Menjalankan Server Inferensi FastAPI

Server memuat ketiga artefak ONNX sekali saat startup ($\approx 7,6$ GB di RAM), lalu menyediakan `/health` dan `/predict`.

```
cd smartCityReport
source .venv/bin/activate
uvicorn serve_model:app --host 0.0.0.0 --port 8000 --workers 1
```

Penting. Gunakan `--workers 1`: setiap worker memuat penuh $\approx 7,6$ GB. *Cold start* lambat (memuat *text encoder* 6,4 GB) — lakukan satu `/health` dan satu `/predict` pemanasan sebelum demo.

7.1 Uji *health*

```
curl -s http://127.0.0.1:8000/health | python3 -m json.tool
# -> {"status": "ok", "image_encoder": "dinov3_large",
#     "text_encoder": "mE5_large", "classes": [...9...], "gpu_available":
#     ↪ false}
```

7.2 Uji *predict* (multipart)

```
curl -s -X POST http://127.0.0.1:8000/predict \
  -F "image=@/path/to/photo.jpg" \
  -F "laporan=ada lubang besar di Jalan Sudirman" \
  -F "latitude=-6.2088" -F "longitude=106.8456" | python3 -m json.tool
# -> predicted_dinas, confidence, all_probabilities,
#     review_required, assignment{agency_name, distance_meters, ...}
```

8 Tahap F — Kontainerisasi Docker (opsional, GPU)

Jalur ini memakai Dockerfile berbasis CUDA 12.1 + onnxruntime-gpu; membutuhkan host ber-GPU NVIDIA (mis. pod RunPod), bukan macOS.

```
cd smartCityReport
docker compose up --build          # build image + mount artifacts/
    ↪ export (read-only)
# image: crm-multimodal:latest, port 8000, GPU direservasi via compose.
    ↪ yaml
```

Catatan artefak. `compose.yaml` me-mount `artifacts/export` sebagai volume *read-only* sehingga artefak 7,6 GB tidak perlu di-*bake* ke dalam image.

9 Tahap G — Publikasi Server via ngrok

Agar perangkat Android dapat menjangkau server melalui internet:

```
brew install ngrok                  # atau unduh dari ngrok.com
ngrok config add-authtoken <TOKEN> # sekali saja (akun gratis)
ngrok http 8000
# -> Forwarding https://abc123.ngrok-free.app -> http://localhost:8000
```

Verifikasi dari luar jaringan:

```
curl -s https://abc123.ngrok-free.app/health
```

Peringatan. URL ngrok gratis berubah setiap restart. Catat URL terbaru dan teruskan ke aplikasi (Tahap H) sebelum demo.

10 Tahap H — Menjalankan Aplikasi Flutter

Aplikasi membaca alamat server dari `dart-define` `CRM_API_URL` (default `http://10.0.2.2:8000` untuk emulator Android).

```
cd smartCityReport/smart_city_reporter_app
flutter pub get
flutter run \
  --dart-define=CRM_API_URL=https://abc123.ngrok-free.app \
  --dart-define=SUPABASE_URL=<supabase_url> \
  --dart-define=SUPABASE_ANON_KEY=<anon_key>
```

10.1 Alternatif tanpa ngrok

- **Emulator:** biarkan default `http://10.0.2.2:8000` (alias localhost host).
- **HP via USB:** `adb reverse tcp:8000 tcp:8000`, lalu `CRM_API_URL=http://localhost:8000`.

11 Tahap I — Verifikasi End-to-End

1. Di aplikasi: ambil foto → tulis narasi → izinkan lokasi → kirim.
2. Layar tinjauan AI menampilkan **instansi prediksi, confidence, top-3, instansi terdekat** (hasil nyata dari server).

3. Koreksi kategori bila perlu → simpan.
4. Laporan tersimpan di Supabase (tabel `reports`) + foto di `bucket report-images`.
5. Operator Instansi memverifikasi; status `resolved` mewajibkan foto bukti penyelesaian.

12 Troubleshooting

Gejala	Penyebab / Solusi
Server OOM / lambat saat start	RAM kurang untuk 7,6 GB; gunakan <code>--workers 1</code> , tutup aplikasi lain.
Invalid image pada <code>/predict</code>	Berkas bukan gambar valid; kirim JPG/PNG via <code>-F "image=@..."</code> .
Aplikasi tak terhubung	URL ngrok kedaluwarsa; perbarui <code>CRM_API_URL</code> dan <code>flutter run</code> ulang.
Emulator gagal localhost	Gunakan 10.0.2.2 (emulator) atau <code>adb reverse</code> (HP fisik).
Paritas ONNX gagal ($> 10^{-5}$)	Periksa <code>opset</code> / <code>operator</code> ekspor pada <code>08_export_onnx.ipynb</code> .
Email verifikasi tidak masuk	SMTP bawaan Supabase ber- <i>rate limit</i> ; gunakan SMTP khusus untuk volume lebih besar.

Table 2: Masalah umum dan penanganannya.

13 Lampiran — Peta Berkas Penting

```

smartCityReport/
  serve_model.py           # FastAPI: /health, /predict, routing
    ↪ Haversine
  Dockerfile, compose.yaml # jalur kontainer GPU
  requirements-serve.txt   # subset penyajian (CPU)
  requirements.txt         # pelatihan (CUDA)
  notebooks/01..09_*.ipynb # analisis -> pelatihan -> ekspor
  src/crm/encoders/{image,text}.py # pembungkus encoder ONNX
  src/crm/fusion.py, pgs.py  # early fusion + kalibrasi PGS
  artifacts/export/         # artefak ONNX yang dilayani
  smart_city_reporter_app/  # aplikasi Flutter (klien)
    lib/core/config/app_config.dart # CRM_API_URL, SUPABASE_URL,
    ↪ ANON_KEY

```